

Amazon Data Firehose Hands-On Lab

To Begin with the Lab

Summary of the Lab

In this lab, **Amazon Data Firehose** was created to deliver streaming data into **Amazon S3**. The video explained that Firehose can either take data from **Amazon Kinesis Data Streams** or use **Direct PUT** as the source. For this practical lab, **Direct PUT** was selected so the stream could receive data directly without needing Kinesis Data Streams first. An S3 bucket was created as the destination, buffer settings were configured so Firehose would collect data for a short period before delivering it, and then the built-in **demo data producer** was used to test the flow. After the test ran, the streamed records appeared in the S3 bucket in folder-like prefixes, and the downloaded file could be opened to verify the data. The lab proved that Firehose can collect incoming data, buffer it, and write it to S3 automatically.

Understanding Amazon Data Firehose

Amazon Data Firehose is a fully managed service that receives streaming data and delivers it to destinations such as **Amazon S3**. It exists so you do not need to build your own streaming ingestion pipeline for simple delivery use cases. In this lab, Firehose acted as the middle service between the producer and S3. The producer sent data to Firehose, Firehose buffered the data, and then Firehose delivered it to S3 after the buffer interval or buffer size was reached.

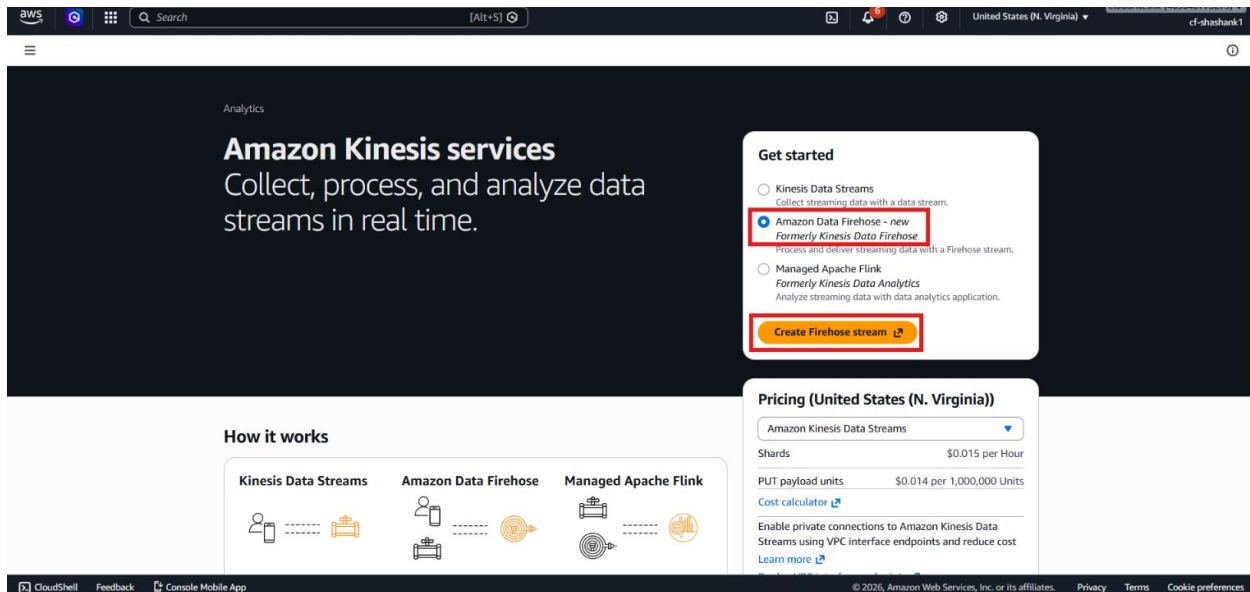
Example:

A stock ticker app sends small updates such as symbol, price, and timestamp. Firehose collects those updates for a few seconds and then writes them to an S3 bucket so the data can later be analyzed or archived.

Lab Steps

Step 1 – Open Amazon Data Firehose

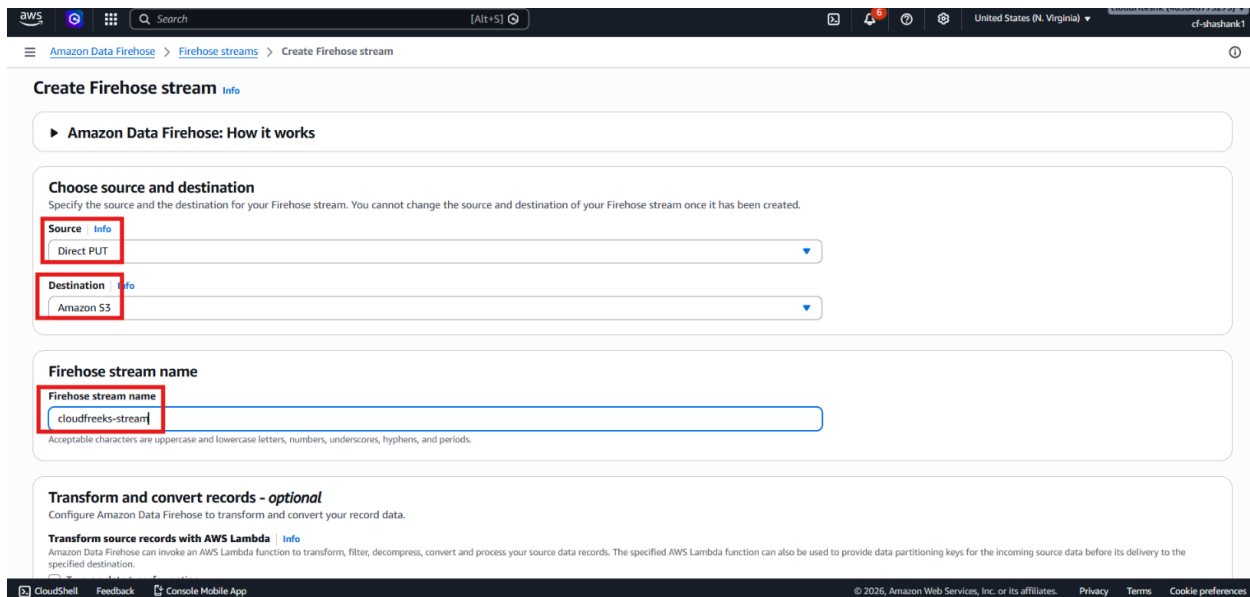
- Sign in to the AWS Management Console.
- Open **Amazon Kinesis**.
- Select **Amazon Data Firehose**.



- Click **Create Firehose stream**.

Step 2 – Choose the Source

- Select **Direct PUT** as the source.
- Understand that Firehose can also use **Kinesis Data Streams** as a source, but that was not used in this lab.
- Enter the Firehose stream name: **cloudfreeks-stream**



Step 3 – Choose the Destination

- Select **Amazon S3** as the destination.
- Create a new S3 bucket if needed.

Destination settings [Info](#)

Specify the destination settings for your Firehose stream.

S3 bucket

Choose a bucket or enter a bucket URI

Format: s3://bucket

New line delimiter

You can configure your Firehose stream to add a new line delimiter between records in objects that are delivered to Amazon S3.

☒ Not enabled

☐ Enabled

Dynamic partitioning [Info](#)

Dynamic partitioning enables you to create targeted data sets by partitioning streaming S3 data based on partitioning keys. You can partition your source data with inline parsing and/or the specified AWS Lambda function. You can enable dynamic partitioning only when you create a new Firehose stream. You cannot enable dynamic partitioning for an existing Firehose stream. Enabling dynamic partitioning incurs additional costs per GiB of partitioned data. For more information, see [Amazon Data Firehose pricing](#).

☒ Not enabled

☐ Enabled

S3 bucket prefix - optional

By default, Amazon Data Firehose appends the prefix "YYYY/MM/dd/HH" (in UTC) to the data it delivers to Amazon S3. You can override this default by specifying a custom prefix that includes expressions that are evaluated at runtime.

Enter a prefix

You can repeat the same keys in your S3 bucket prefix. Maximum S3 bucket prefix characters: 1024.

S3 bucket error output prefix - optional

You can specify an S3 bucket error output prefix to be used in error conditions. This prefix can include expressions for Amazon Data Firehose to evaluate at runtime.

Enter a prefix

- Enter the bucket name: cloudfreeks-stream-data

General configuration

AWS Region

US East (N. Virginia) us-east-1

Bucket type [Info](#)

☒ **General purpose**

Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

☐ **Directory**

Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket namespace

Choose the namespace where you want to create your bucket. [Learn more](#)

☒ **Global namespace**

By default, S3 creates general purpose buckets in the global namespace.

☐ **Account Regional namespace (recommended)**

General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.

Bucket name [Info](#)

cloudfreeks-stream-data

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#)

Copy settings from existing bucket - optional

Only the bucket settings in the following configuration are copied.

Format: s3://bucket/prefix

Object Ownership [Info](#)

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

Object Ownership

- Browse and select the bucket.
- Keep the destination configuration simple.

Destination settings Info

Specify the destination settings for your Firehose stream.

S3 bucket

s3://cloudfreeks-stream-data Browse Create

Format: s3://bucket

New line delimiter

You can configure your Firehose stream to add a new line delimiter between records in objects that are delivered to Amazon S3.

☒ Not enabled
☐ Enabled

Dynamic partitioning Info

Dynamic partitioning enables you to create targeted data sets by partitioning streaming S3 data based on partitioning keys. You can partition your source data with inline parsing and/or the specified AWS Lambda function. You can enable dynamic partitioning only when you create a new Firehose stream. You cannot enable dynamic partitioning for an existing Firehose stream. Enabling dynamic partitioning incurs additional costs per GiB of partitioned data. For more information, see [Amazon Data Firehose pricing](#).

☒ Not enabled
☐ Enabled

S3 bucket prefix - optional

By default, Amazon Data Firehose appends the prefix "YYYY/MM/dd/Hh" (in UTC) to the data it delivers to Amazon S3. You can override this default by specifying a custom prefix that includes expressions that are evaluated at runtime.

Enter a prefix

You can repeat the same keys in your S3 bucket prefix. Maximum S3 bucket prefix characters: 1024.

S3 bucket error output prefix - optional

You can specify an S3 bucket error output prefix to be used in error conditions. This prefix can include expressions for Amazon Data Firehose to evaluate at runtime.

Enter a prefix

Step 4 – Configure Buffer Settings

- Leave transformation and conversion options disabled.
- Set the buffer size to **1 MiB**.
- Set the buffer interval to **5 seconds**.
- Choose the appropriate timezone, such as **Asia/Kolkata** for India.

S3 bucket and S3 error output prefix time zone Info

Choose a time zone that you want to use for date and time in S3 prefixes

Asia/Calcutta

▼ Buffer hints, compression, file extension and encryption

The fields below are pre-populated with the recommended default values for S3. Pricing may vary depending on storage and request costs.

S3 buffer hints

Amazon Data Firehose buffers incoming records before delivering them to your S3 bucket. Record delivery is triggered once the value of either of the specified buffering hints is reached.

Buffer size

The higher the buffer size may be lower in cost with higher latency. The lower buffer size will be faster in delivery with higher cost and less latency.

1 MiB

Minimum: 1 MiB, maximum: 128 MiB. Recommended: 5 MiB.

Buffer interval

The higher the interval allows more time to collect data and the size of data may be bigger. The lower interval sends the data more frequently and may be more advantageous when looking at shorter cycles of data activity.

5 seconds

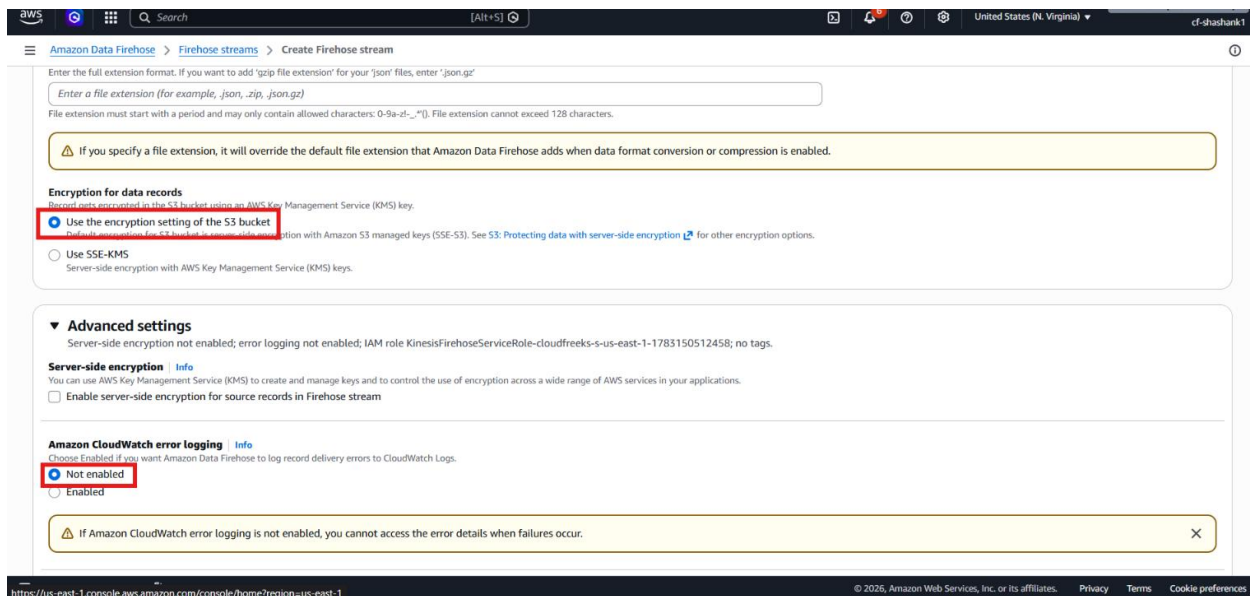
Minimum: 0 seconds, maximum: 900 seconds. Recommended: 300 seconds.

Compression for data records

Amazon Data Firehose can compress records before delivering them to your S3 bucket.

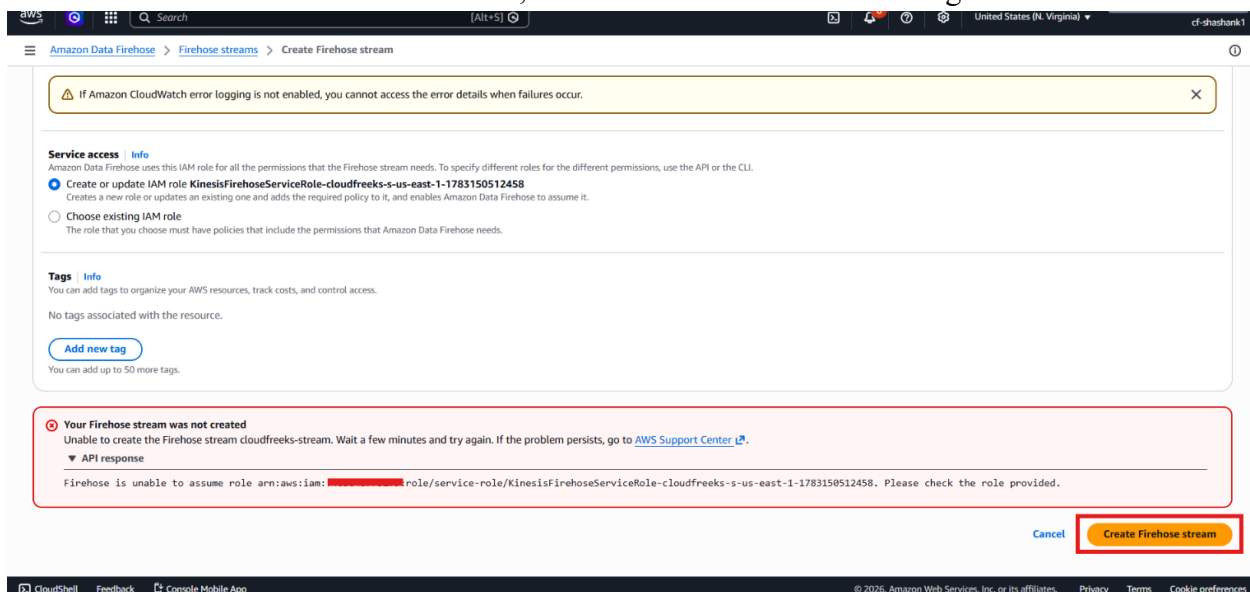
☒ Not enabled
☐ GZIP
☐ Snappy
☐ Zip
☐ Hadoop-Compatible Snappy

- Keep encryption as the default S3 encryption.
- Leave CloudWatch error logging disabled for this lab.



Step 5 – Create the Firehose Stream

- Click **Create Firehose stream**.
- If AWS shows an error the first time, click **Create Firehose stream** again.



- Wait until the stream becomes **Active**.

Amazon Data Firehose > Firehose streams > cloudfreeks-stream

Creating cloudfreeks-stream
It can take up to 2 minutes before the status is updated.

cloudfreeks-stream Info [Delete Firehose stream](#)

Amazon CloudWatch error logging is not enabled
Enabling Amazon CloudWatch error logging is strongly recommended. This practice ensures that you can access error details when failures occur. [Edit Amazon CloudWatch error logging](#) X

Firehose stream details

Status Creating	Destination Amazon S3	Data transformation Not enabled	Creation time July 04, 2026 at 13:19 GMT+5:30
Source Direct PUT	ARN arn:aws:firehose:us-east-1: [redacted] :deliverystream/cloudfreeks-stream	Dynamic partitioning Not enabled	

Test with demo data Info
Ingest simulated data to test the configuration of your Firehose stream. Standard Amazon Data Firehose charges apply.

Monitoring Configuration

Firehose stream metrics Info

Step 6 – Test with Demo Data

- Open Test with demo data.

Amazon Data Firehose > Firehose streams > cloudfreeks-stream

cloudfreeks-stream Info [Delete Firehose stream](#)

Amazon CloudWatch error logging is not enabled
Enabling Amazon CloudWatch error logging is strongly recommended. This practice ensures that you can access error details when failures occur. [Edit Amazon CloudWatch error logging](#) X

Firehose stream details

Status Active	Destination Amazon S3	Data transformation Not enabled	Creation time July 04, 2026 at 13:19 GMT+5:30
Source Direct PUT	ARN arn:aws:firehose:us-east-1: [redacted] :deliverystream/cloudfreeks-stream	Dynamic partitioning Not enabled	

Test with demo data Info
Ingest simulated data to test the configuration of your Firehose stream. Standard Amazon Data Firehose charges apply.

Monitoring Configuration

Firehose stream metrics Info

[Investigate with AI - new](#) 1h 3h 12h 1d 3d 1w Custom UTC timezone [Explore related](#)

Incoming bytes Incoming put requests Incoming records

- Click Start sending demo data.

The screenshot shows the Amazon Data Firehose console for a stream named 'cloudfreeks-stream'. The 'Test with demo data' section is active, displaying a JSON script for ingesting simulated data. Below the script, 'Step 1' is titled 'Start sending demo data to your Firehose stream'. A button labeled 'Start sending demo data' is highlighted with a red rectangle. 'Step 2' is titled 'Stop sending demo data to your Firehose stream after you've concluded your test to stop incurring usage charges' and includes a 'Stop sending demo data' button. The 'Monitoring' tab is selected, showing 'Firehose stream metrics'.

- Let Firehose send data for a few seconds.
- Click **Stop sending demo data**.

This screenshot shows the same Amazon Data Firehose console after the test has been initiated. The 'Test with demo data' section now shows a status of 'Sending demo data' next to the 'Start sending demo data' button. The 'Stop sending demo data' button in 'Step 2' is highlighted with a red rectangle. The 'Monitoring' tab remains selected, showing the 'Firehose stream metrics'.

- This simulates a producer sending records into the Firehose stream.

Step 7 – Verify the Data in S3

- Open the S3 bucket used as the destination.
- Refresh the bucket.
- Look for a generated folder prefix, such as a timestamp-based folder.

The screenshot shows the Amazon S3 console interface. On the left, the navigation pane includes 'Buckets', 'Files', 'Access management and security', and 'Storage management and insights'. The main content area displays the 'cloudfreeks-stream-data' bucket. Under the 'Objects (1)' section, a table lists the contents. A folder named '2026/' is highlighted with a red box. Above the table, the 'Download' button is also highlighted with a red box.

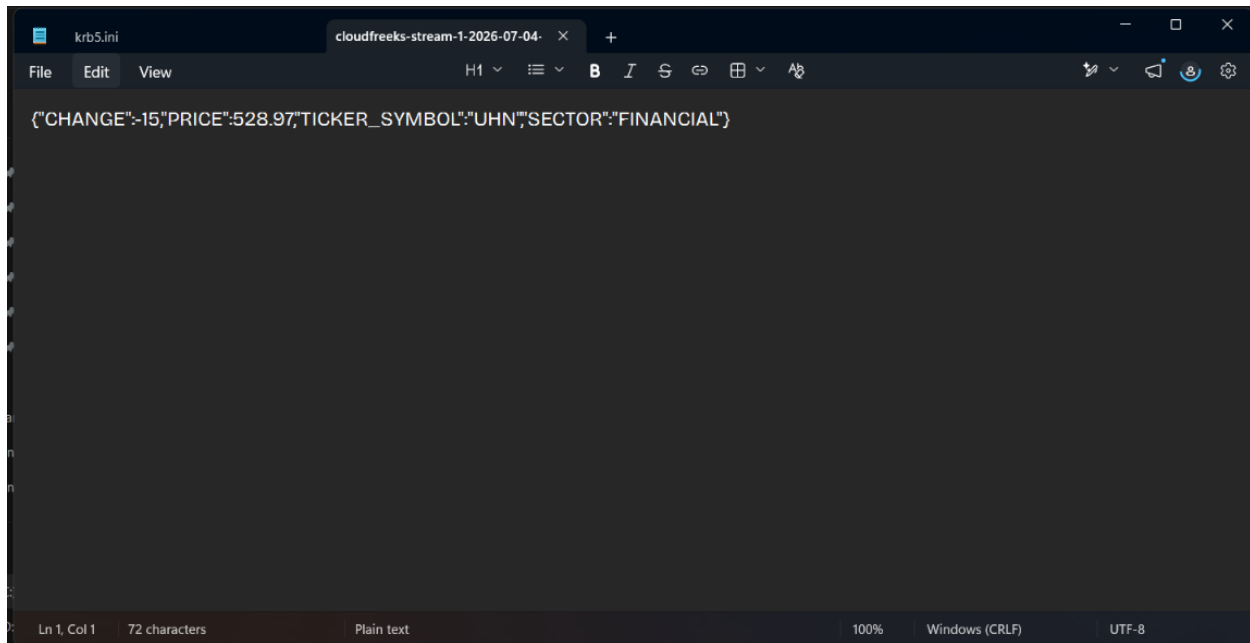
Name	Type	Last modified	Size	Storage class
2026/	Folder	-	-	-

- Keep opening folders if there are multiple ones till you find file.
- Open the created file.

The screenshot shows the Amazon S3 console interface, now displaying the '13/' folder. The breadcrumb navigation at the top shows the path: 'cloudfreeks-stream-data > 2026/ > 07/ > 04/ > 13/'. The 'Download' button is highlighted with a red box. The table below lists two objects, with the first one highlighted by a red box.

Name	Type	Last modified	Size	Storage class
cloudfreeks-stream-1-2026-07-04-13-27-22-713ef32e-9e6d-4d27-b8ee-aac58fe9dd5f	-	July 4, 2026, 13:27:28 (UTC+05:30)	72.0 B	Standard
cloudfreeks-stream-1-2026-07-04-13-27-28-e091fbb8-8fb3-4dbe-a32d-bb62c11a2398	-	July 4, 2026, 13:27:36 (UTC+05:30)	75.0 B	Standard

- Download it and open it in **Notepad** or another text editor.

A screenshot of a code editor window. The title bar shows two tabs: 'krb5.ini' and 'cloudfreeks-stream-1-2026-07-04'. The editor has a dark theme. The main text area contains a single line of JSON: `{"CHANGE":-15,"PRICE":528.97,"TICKER_SYMBOL":"UHN","SECTOR":"FINANCIAL"}`. The status bar at the bottom indicates 'Ln 1, Col 1', '72 characters', 'Plain text', '100%', 'Windows (CRLF)', and 'UTF-8'.

```
cloudfreeks-stream-1-2026-07-04
{"CHANGE":-15,"PRICE":528.97,"TICKER_SYMBOL":"UHN","SECTOR":"FINANCIAL"}
```

- Confirm that the file contains the streamed data, such as ticker values or sample records.

Verification

- An **Amazon Data Firehose** stream was created successfully.
- The source was set to **Direct PUT**.
- The destination was configured as **Amazon S3**.
- Buffer size and buffer interval were configured correctly.
- Demo data was sent through Firehose.
- Records appeared in the S3 bucket.
- The downloaded file contained the expected streamed data.